8. Poročilo:
Metoda končnih elementov: lastne rešitve

Modelska analiza II, 2023/24

Aljaž Ravnikar, dipl. inž. str. (UN), 23232128

22. julij 2025

1 Homogena kvadratna opna

Najprej sem metodo končnih elementov (MKE) preizkusil na homogeni kvadratni domeni v dveh dimenzijah. Rešujemo problem lastnih vrednosti

$$\nabla^2 u + k^2 u = 0, \tag{1.1}$$

ki ga diskretiziramo z MKE

$$\boldsymbol{K} \cdot \boldsymbol{U} + k^2 \boldsymbol{M} \cdot \boldsymbol{u} = 0, \tag{1.2}$$

na globalno togostno $\boldsymbol{K}$ in masno $\boldsymbol{M}$ matriko. Pri tem uporabimo linearne trikotne končne elemente (KE), ki jih definiramo s tremi vozlišči i, j, k. Za vsak element definiramo lokalni togostni in masni matriki

$$\boldsymbol{K}_{ijk} = \frac{1}{4\,S_{ijk}} \left[(x_i - x_k)(x_k - x_j) + (y_i - y_k)(y_k - y_j) \right], \tag{1.3}$$

$$\boldsymbol{M}_{ijk} = \frac{S_{ijk}}{12}\, \boldsymbol{I}, \tag{1.4}$$

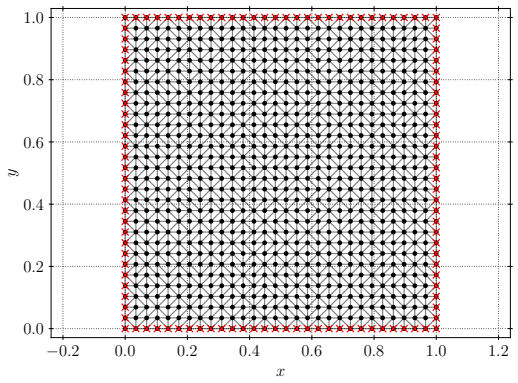
ki ju sestavimo v globalni matriki $\boldsymbol{K}$ in $\boldsymbol{M}$, pri čemer upoštevamo indekse vozlišč v vektorju neznank $\boldsymbol{U}$. Pri tem je S_{ijk} ploščina trikotnika, ki ga tvorijo vozlišča i, j, k.
$->$ Opomba: Lokalna masna matrika je diagonalna, globalna pa le pasovna!

Robni pogoji so Diricheltovi $U_i = 0$, kar lahko zagotovimo tako, da izločimo vrstice in stolpce globalnih matrik, ki ustrezajo vozliščem na robu domene. Ker je brisanje vrstic in stolpcev zoprno z vidika spreminjajočih se indeksov, sem uporabil način, da definiram v vrstici in stolpcu togostne matrike z indeksom i, ki ustreza vozlišču na robu domene vrednost nič, le na mestu K_{ii} določim visoko togost (v tem primeru 1e6), kar onemogoči premike, vendar previsoka vrednosti lahko povzroča numerične nestabilnosti pri reševanju sistema. Masno matriko pustim nedotaknjeno.

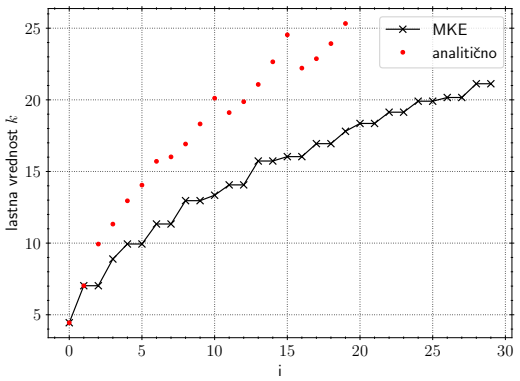
V tem primeru sem izračunane lastne vrednosti in oblike z MKE primerjal z analitičnimi

$$\mathcal{U}_{mn}(x, y) = \sin\left(\frac{m\pi x}{L}\right) \sin\left(\frac{n\pi y}{L}\right), \tag{1.5}$$
$$k = \sqrt{(m\,\pi)^2 + (n\,\pi)^2}, \quad m, n = 1, 2, 3 \ldots$$

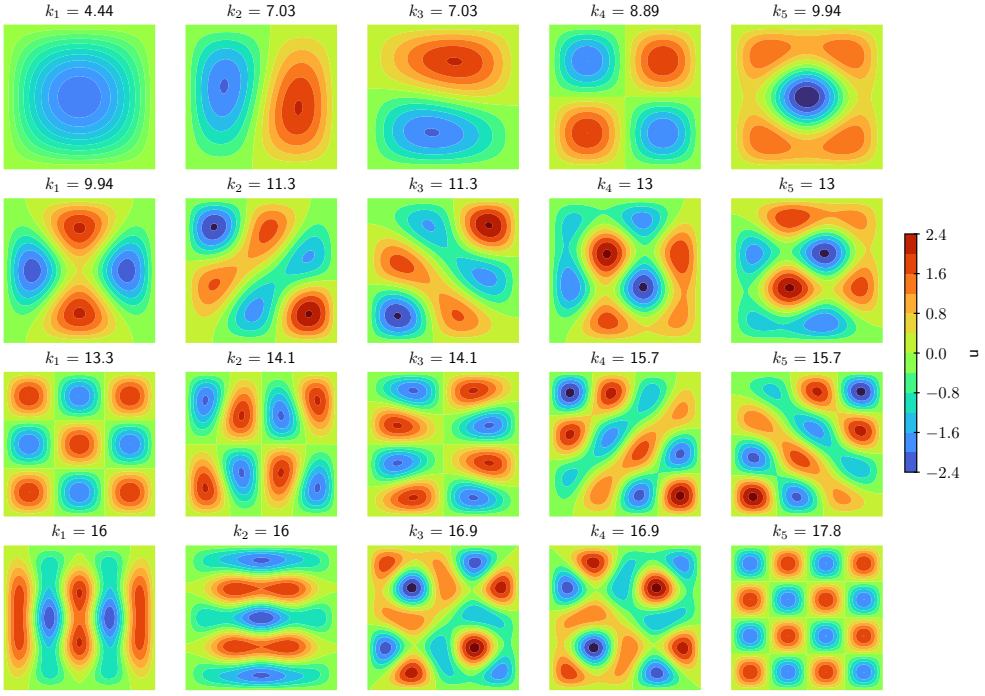
Mrežo točk generiram enakomerno in z Delaunay triangulacijo, kar je razvidno na spodnji sliki. Opazim, da sta prvi dve lastni vrednosti zelo blizu analitičnim, sledeče pa so že precej oddaljene, vendar se z zgoščevanjem mreže stanje izboljšuje. Lastne oblike so smiselne tudi pri višjih lastnih vrednostih, odstopanja od analitičnih so predvsem v amplitudah.



Slika 1.1: Mreža za kvadratno domeno z 900 vozlišči



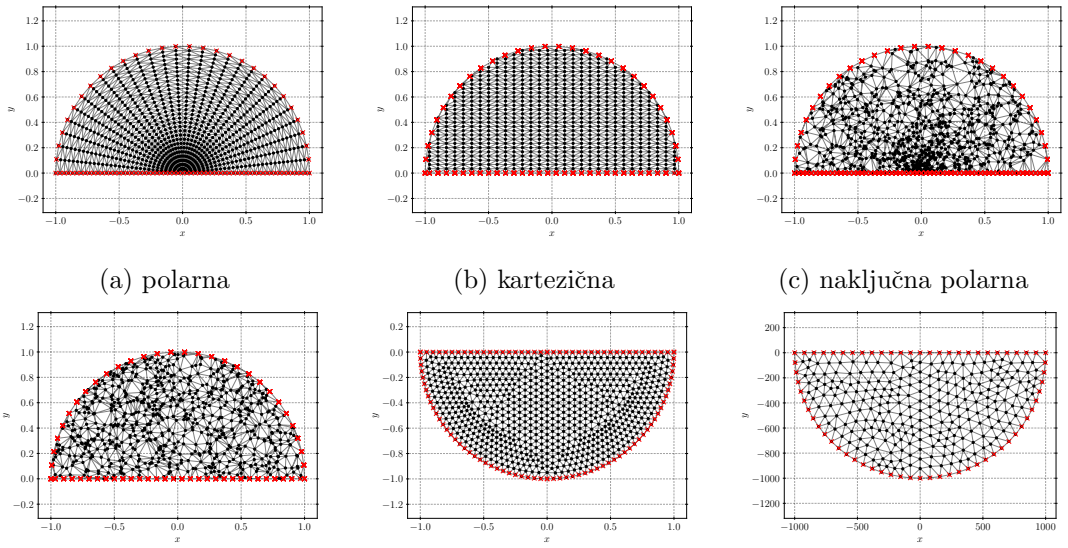
Slika 1.2: Izračunane lastne vrednosti



Slika 1.3: Rezultati lastnih oblik z 10 000 vozliščno mrežo

2 Homogena polkrožna opna

Sedaj preizkusimo metodo na polkrožni domeni. Pri tem lahko uporabimo različne načine tvorbe točk, kar je prikazano na spodnji sliki. V primeru polarnega koor. sistema je v središču velika zgostitev točk, pri karteziničnem pa lahko prihaja do neenakomernih velikosti KE na robovih domene. Prav tako sem poizkusil na mrežah iz komercialnih programov za MKE, kot sta Ansys in ABAQUS, kjer določimo le število točk na robu.



(a) polarna (b) kartezična (c) naključna polarna

(d) naključna kartezična (e) Abaqus (f) Ansys

Slika 2.1: Primeri mreže za polkrožno domeno z različno razporeditvijo točk

Tokrat je analitična rešitev drugačna v radialni smeri

$$\mathcal{U}_{mn}(r,\varphi) = J_m\left(k_m^n\, r\right)\, \sin(m\varphi), \quad m,n = 1,2,3\ldots, \tag{2.1}$$

kjer je k_m^n n-ta ničla J_m Besselove funkcija prve vrste reda m. Način reževanja z MKE pa identičen kot pri kvadratni domeni, le da moramo upoštevati robne pogoje na mreži v polkrožni domeni!

2.1 Galerkinov nastavek

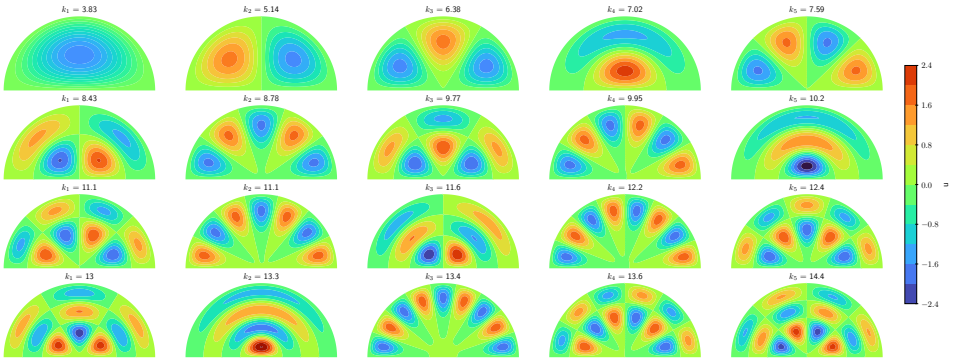
Namesto linearnih baznih vinkcij, kot pri MKE, vzamemo bolj primerne funcije za radialno smer (potenčne funkcije)

$$\Phi_{mn}(r,\phi) = r^{m+n}\,(1-r)\,\sin(m\,\varphi), \quad n = 0,1,2\ldots, \quad m = 1,2,3\ldots \tag{2.2}$$

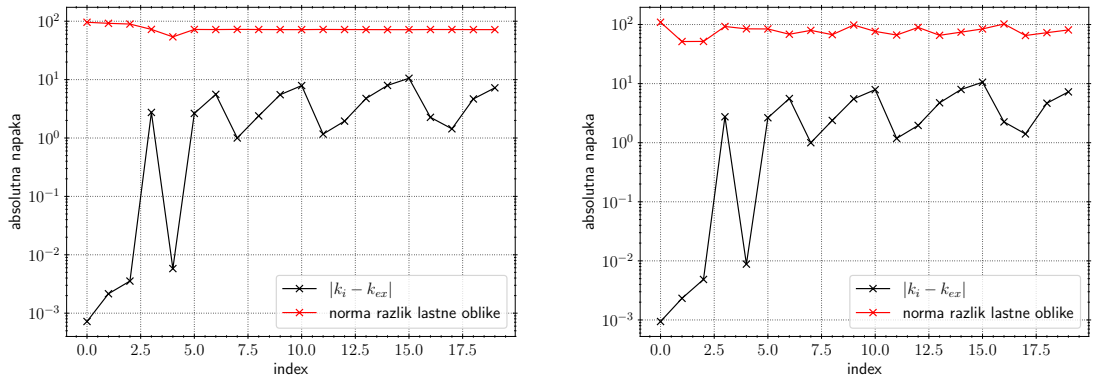
V tem primeru dobimo bločno diagonalno togostno in masno matriko, vendar že s precej manjšim številom točk kot pri MKE lahko dobimo boljšo rešitev.

2.2 Lastne vrednosti in oblike

Na spodnji sliki je prikazano, kako se napake lastnih vrednosti razlikujejo glede na način diskretizacije, kjer na točnost lastnih vrednosti način diskretizacije ne vpliva, dočim bolj vpliva na točnost lastnih oblik, merjenih z normo razlike glede na analitično rešitev. Pri prvih treh lastnih vrednostih je napaka še razmeroma majhna, nato pa naraste, vendar je presenetljivo 5. lastna vrednost povsod precej točna.



Slika 2.2: Lastne oblike polkrožne opne. Kartezična mreža z 8 008 vozlišči



(a) kartezična 8 008 vozlišč

(b) polarna 10 001 vozlišč

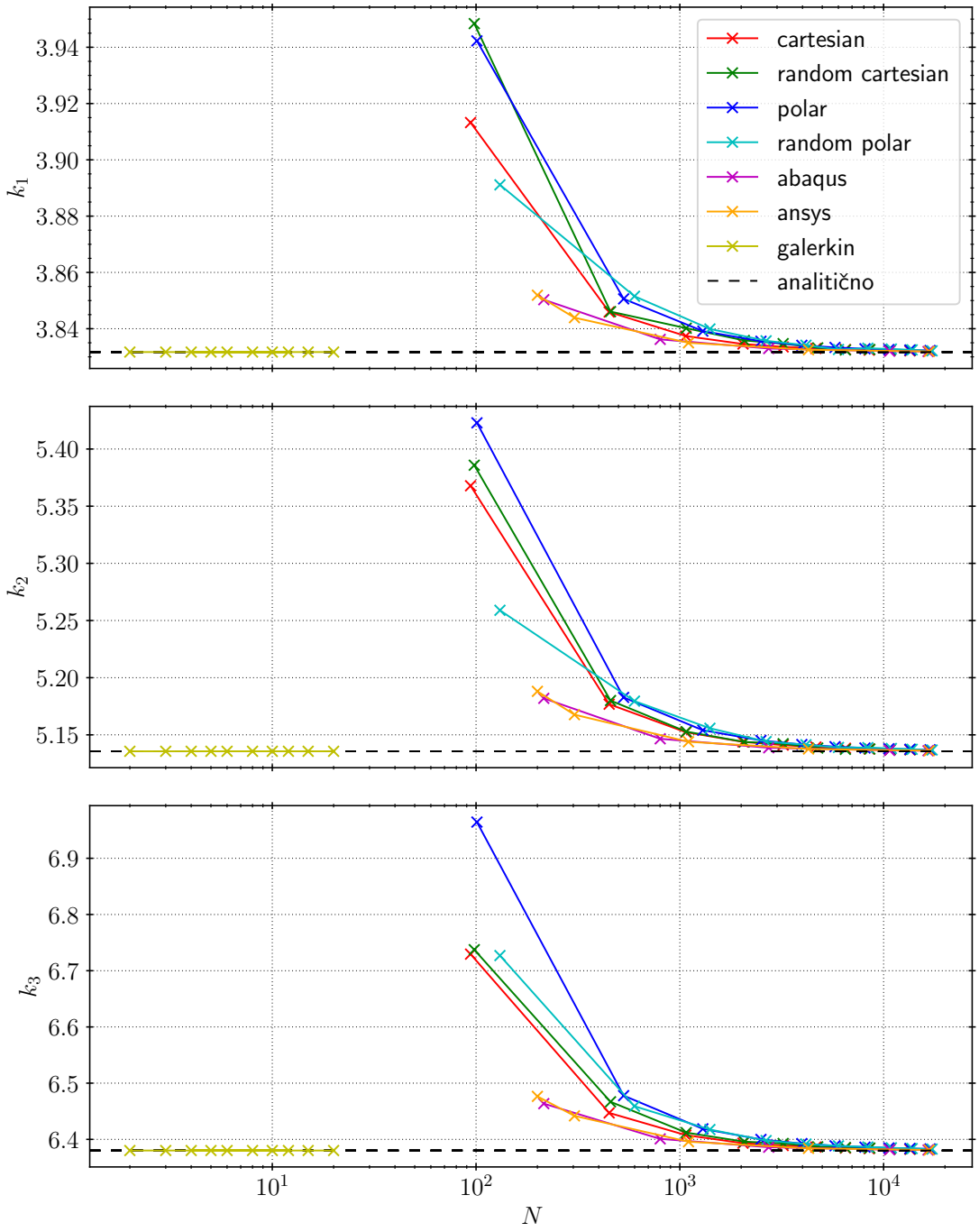
(c) naključna kartezična 8 040 vozlišč

(d) naključna polarna 10 301 vozlišč

Slika 2.3: Napake lastnih vrednosti polkrožne opne v primerjavi z analitično rešitvijo

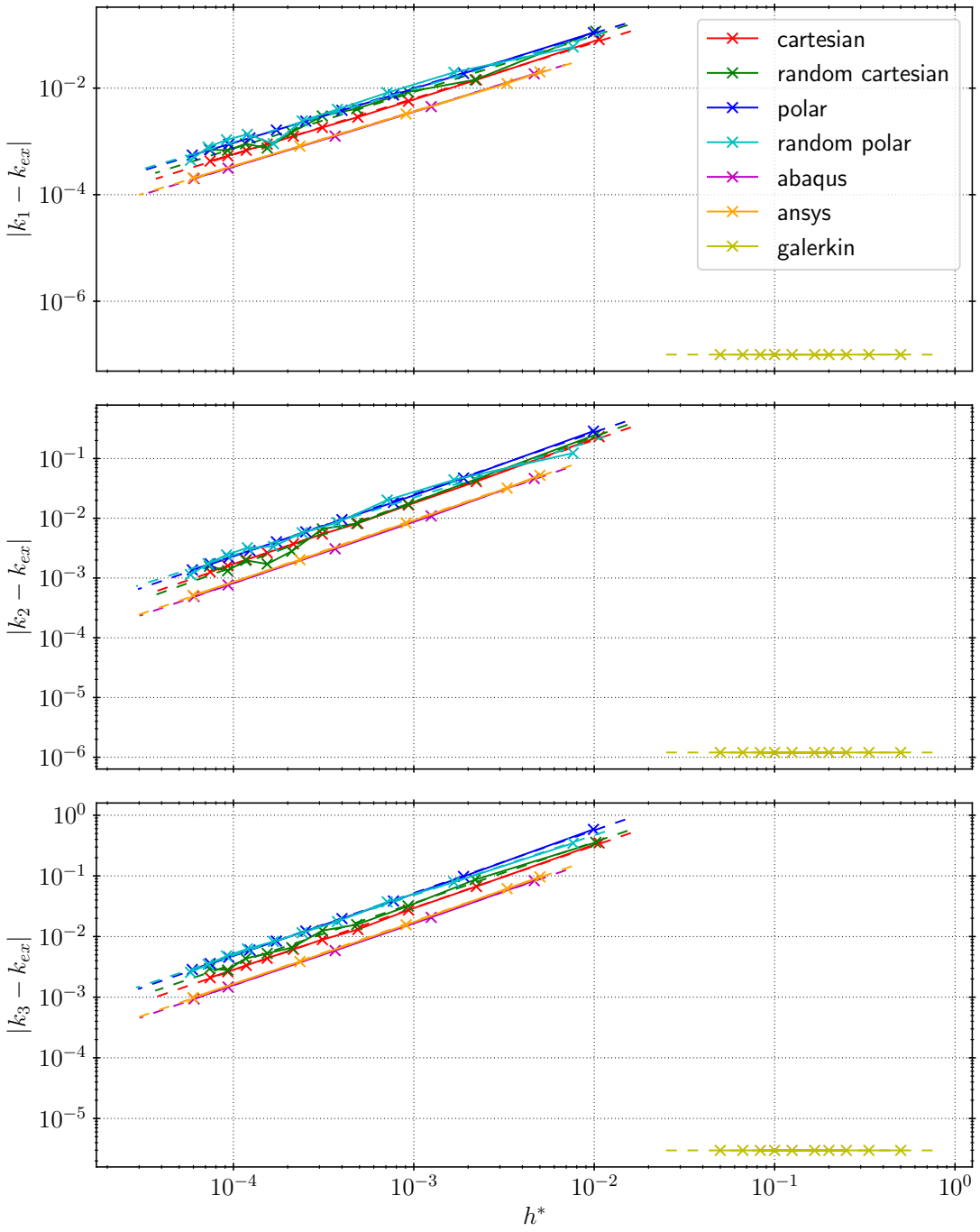
2.3 Konvergenca

Opazim, da prve tri lastne vrednosti konvergirajo k analitični, za višje pa bi potreboval še gostejšo mrežo, da bi dosegel primerljivo natančnost.Velja poudariti, da je Glaerkinova metoda skoraj popolnoma točna že pri nizkem številu računskih točk in se z zgoščevanjem rešitev skoraj ne izboljša.

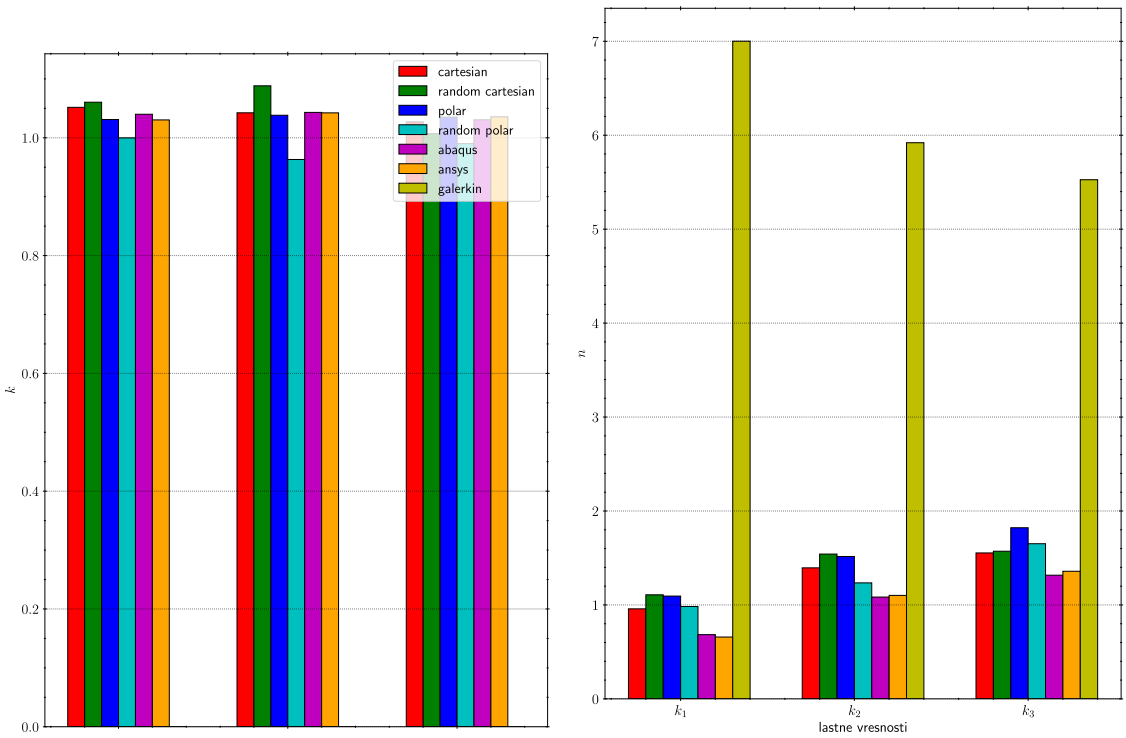


Slika 2.4: Konvergenca prvih treh lastnih vrednosti k analitični rešitvi

Na spodnji sliki je konvergenca napak lastnih vrednosti še bolj nazorno prikazana, hkrati sem ocenil tudi hitrost konvergence tako, da sem v log-log prostoru na podatke 'fital' linearno regresijo, kar je prikazano na sliki 2.6. Zaradi narave konvergence se to ne izkaže kot smiselno pri Galerkinovi metodi, saj je $k \approx 0$ in določamo le n.Pričakovano se najbolje izkažeta mreži komercialnih programov, vendar lastne mreže ne zaostajajo pretiroma. Preseneča me, da se oba koeficienta gibljeta okoli vrednosti ena!



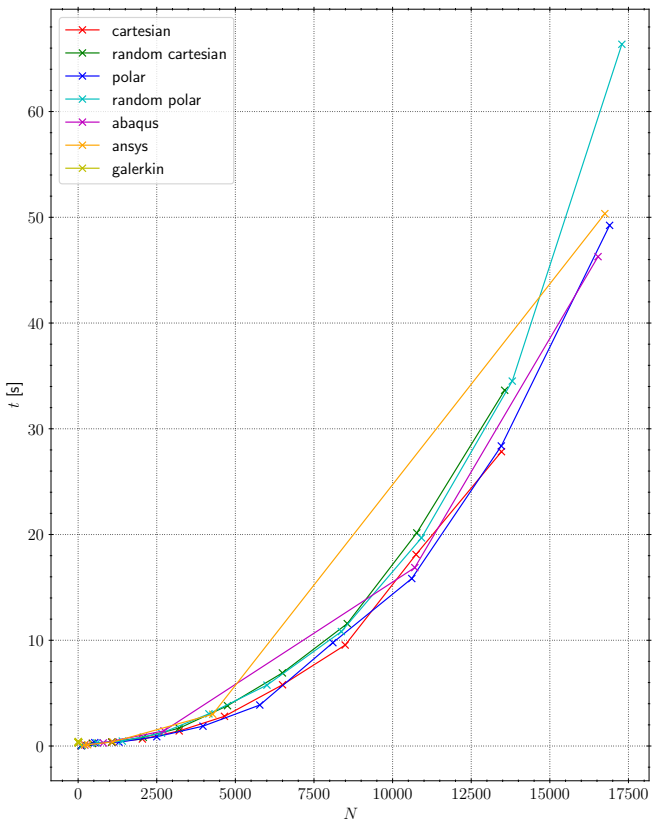
Slika 2.5: Konvergenca napak lastnih vrednosti v odvisnosti od gostote diskretizacije



(a) k (b) n

Slika 2.6: Koficienti linearne regresije za mreže MKE na sliki 2.5

Računski čas pričakovano narašča nad linearno, kar je pričakovano za metodo *Lancoz* iz ARPACK, ki je v ozadju uporabljenega *scipy.sparse.linalg.eigsh*. Pri tem je čas reševanja sistema vseeno precej manjši, kot pri Galerkinovi metodi na enakem številu točk.



Slika 2.7: Računski čas za sliko 2.5

2.3.1 Ekstrapolacija rezultatov

Ker imam v tem primeru že izračunanih več primerov z različno prostorsko diskretizacijo (gostoto mreže), lahko uporabim precej bolj preprost in poceni trik, kako bolje oceniti rezultat, brez da ponovno izvedem izračun. To se zove Richardsonova ekstrapolacija, katere rezultat je tako dober, kot če bi imeli na voljo metodo, ki ima red enak dvakratniku števila izračunov, na katerih smo izvedli ekstrapolacijo.

V dokumentu [1] sem našel opis Richardsonove ekstrapolacije in enačbo

$$A_k(h) = \frac{1}{4^k - 1} \left(4^k A_{k-1}\left(\frac{h}{2}\right) - A_{k-1}(h) \right), \tag{2.3}$$

ki za skalarni veličini $A(\frac{h}{2})$ in $A(h)$ izračuna nov približek. Pri tem je k število vseh izračunov na prostorskih diskretizacijah v razmerju 2, zato lahko na tak način iterativno izboljšamo rezultat na red $O(h^{2\,k})$.

Vendar se na tak način ekstrapoliran tlak razlikuje od onega, predstavljenega pri predmetu *Računalniška dinamika tekočin* na Fakulteti za strojništvo:

$$A = A\left(\frac{h}{2\,r}\right) - \frac{A\left(\frac{h}{r}\right) - A\left(\frac{h}{2\,r}\right)}{r^p - 1}, \quad p = \frac{\ln \frac{A(h) - A\left(\frac{h}{r}\right)}{A\left(\frac{h}{r}\right) - A\left(\frac{h}{2\,r}\right)}}{\ln r}, \tag{2.4}$$

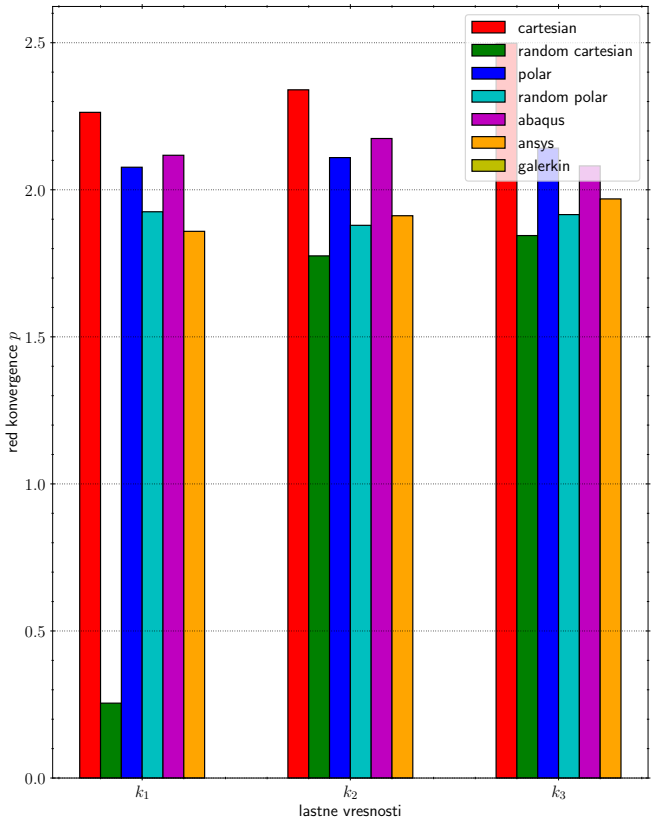
kjer je r faktor zgostitve mreže (v tem primeru štiri) in p red konvergence. Za to enačbo potrebujemo natanko tri mreže s konstantnim faktorjem zgostitve. Pri tem sem uporabil prve tri mreže (najredkejše) iz prejšnjega primera, ki so pravilno zgoščene, pri lastno generiranih mrežah pa sem določil število vozlišč na robovih domene: 16,32,64, kar je sicer manj kot v Ansysu in Abaqusu. Vseeno pa rezultati niso preveč slabši, v nekaterih primerih celo presenetljivo boljši! Pričakovano se naključno generirane mreže slabše obnesejo, saj lahko prihaja do oscilirajoče konvergence.

Glede rešitev opazimo, da je red konvergence p okoli 2 za vse načine, razen naključno generirane kartezične mreže za prvo lastno vrednost, kjer je precej nizek. To je posledica oscilacij zaradi naključno generirane mreže, ko prihaja do natančnejših rezultatov pri redkejših mrežah, kot pri gostejših, saj so zaradi naključne porazdelitve točk nekateri KE precej nepravilnih oblik. Za odpravo takih nepravilnosti bi potebovali boljšo ekstrapolacijo.

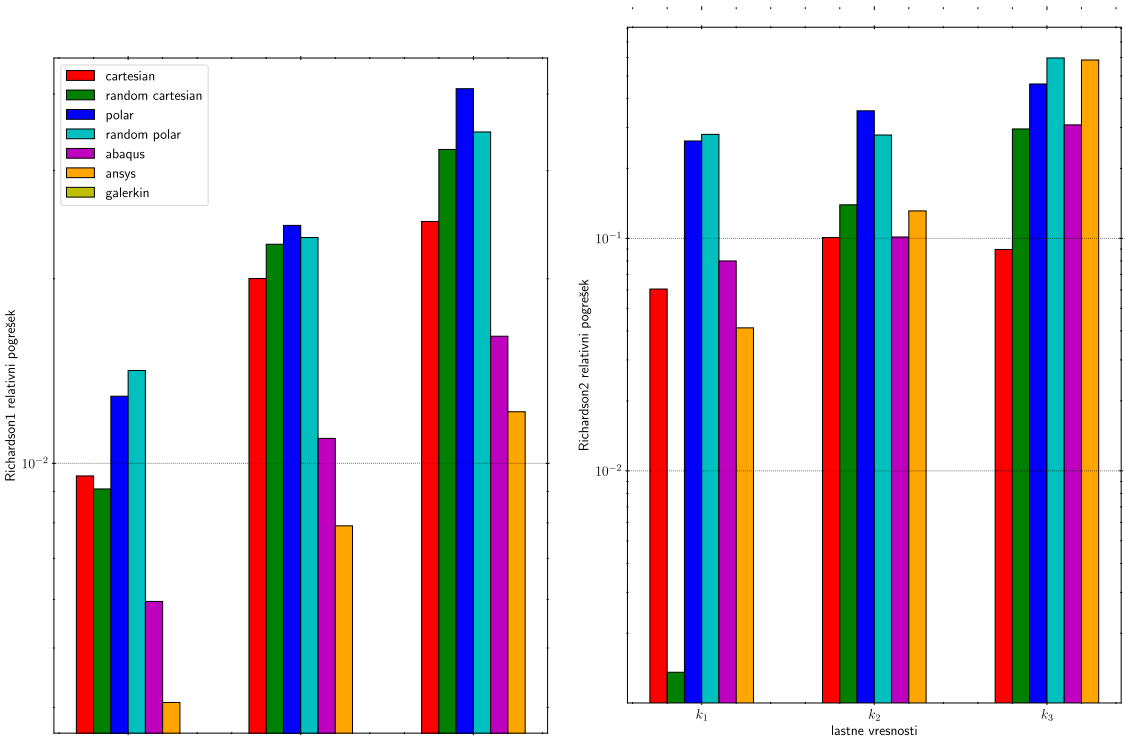
Ravno obratno pa je moč zazanati pri točnosti ekstrapolirane vrednosti glede na analitično, kjer ima 2. način ekstrapolacije precej boljše rezultat natanko za ono vrednost, ki ima izjemno nizek red konvergence!

Pri ekstrapolaciji sicer pričakovano dosežemo manjše pogreške pri nižjih lastnih vrednostih, vendar so ekstrapolirane vrednosti na splošno točnejše po 1., kot po 2. načinu, kjer se vse gibljejo okoli vrednosti pogreška $0,1$, pri 1. načinu pa so vse pod njim. Prav tako imajo komercialne mreže po 1. načinu očitno boljše rezultate, kot lastne generirane mreže.

[1] Duke University, Math 563 Lecture Notes: Differentiation, Richardson Extrapolation, pomlad 202, dostopno na povezavi



Slika 2.8: Red konvergence



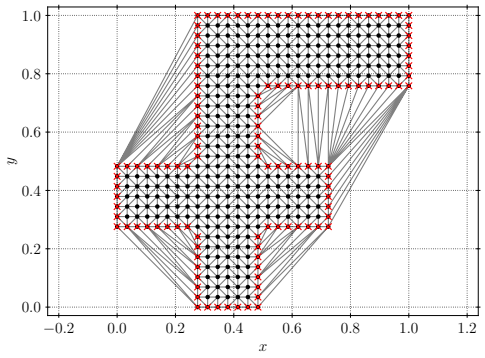
(a) 1.način po 2.3

(b) 2.način po 2.4

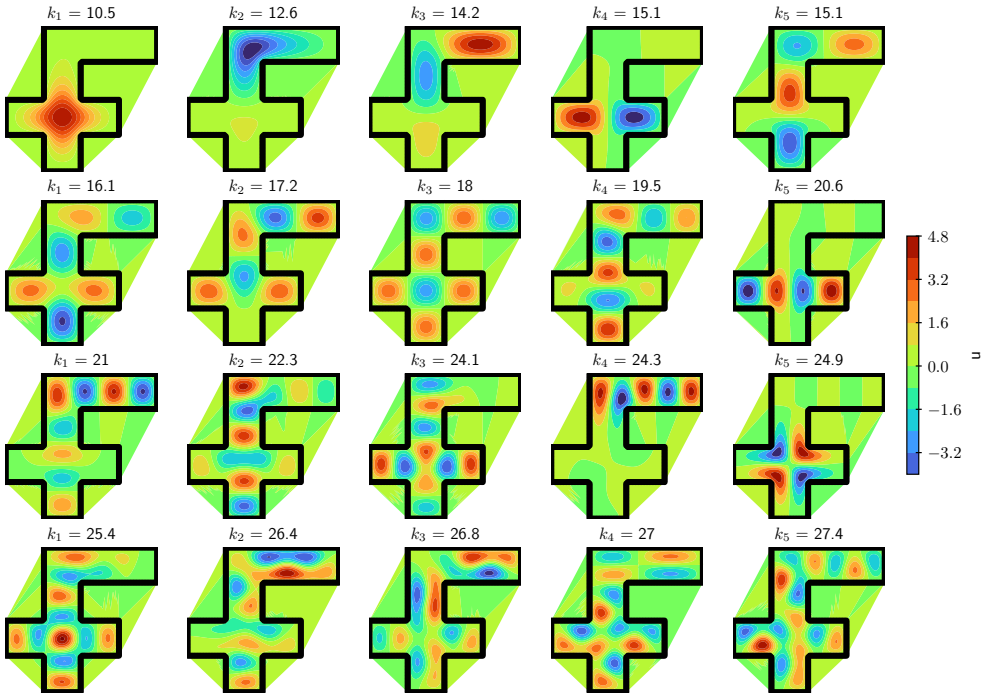
Slika 2.9: Napaka glede na analitično rešitev ekstrapolirane vrednosti po Richardsonu na dva načina

3 Opna ‘f’ iz 6. naloge

Na spodnji sliki je prikazana mreža, ki sem jo uporabil za diskretizacijo opne ‘f’ iz 6. naloge. Mrežo sem generiral podobno kot v prejšnji nalogi, pri čemer sem obdržal le točke v nesenčenem delu in generiral robne točke. Delaunay triangulacija sicer pomreži tudi točke na robu zunaj domene, kar ni pravilno in zaželjeno, vendar imajo takšni končni elementi vsa vozlišča na robu, zato so v primeru Dirichletovega robnega pogoja $U = 0$ vsi pomiki v KE (numerično) enaki nič!



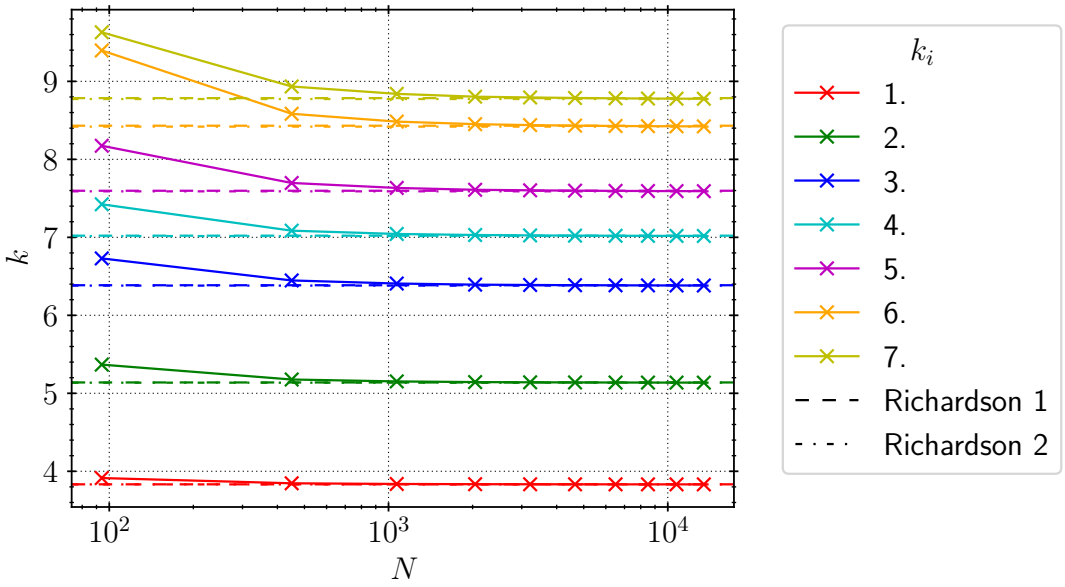
Slika 3.1: Mreža z 435 vozlišči



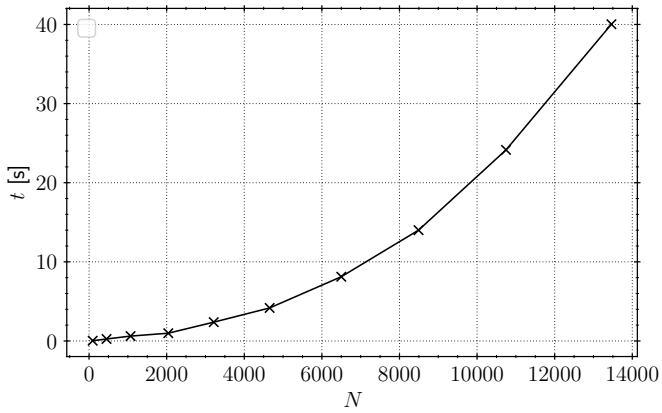
Slika 3.2: Lastne oblike in vrednosti ’f’ opne na mreži z 5000 vozlišči

Opravim konvergenčno analizo prvih 7 lastnih vrednosti in opazim, da se na mrežah z nad 1000 vozlišči rešitev precej približa ekstrapolirani vrednosti. Žal analitične rešitve ne poznamo, zato lahko rezultate primerjam le z gostejšimi mrežami. Red konvergence je precej nizek, ampak primerljiv za vse lastne vrednosti. Zanimivo je, da vse konvergirajo

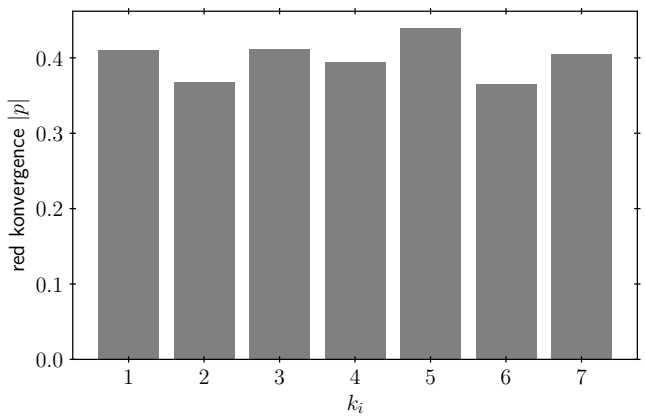
navzdol do ‘prave’ vrednosti. Računski čas pričakovano narašča precej hitro.



Slika 3.3: Konvergenca prvih sedem lastnih vrednosti



Slika 3.4: Računski čas



Slika 3.5: Red konvergence